Implementing Conditional Access on FPGA

A dual-core, cache-coherent RISC-V processor with conditional access [1], on an AMD KR260

Aryan Kumar, Nishant Dhankhar, Kudumula Vinod Kumar Reddy *Advisor:* Ajay Singh

Department of Computer Science and Engineering, IIT Gandhinagar

1 The problem: use-after-free

In a lock-free data structure, threads read shared nodes without taking locks while other threads remove them. If a removed node is freed while another core still holds a pointer to it, that core reads freed memory (Figure 1).

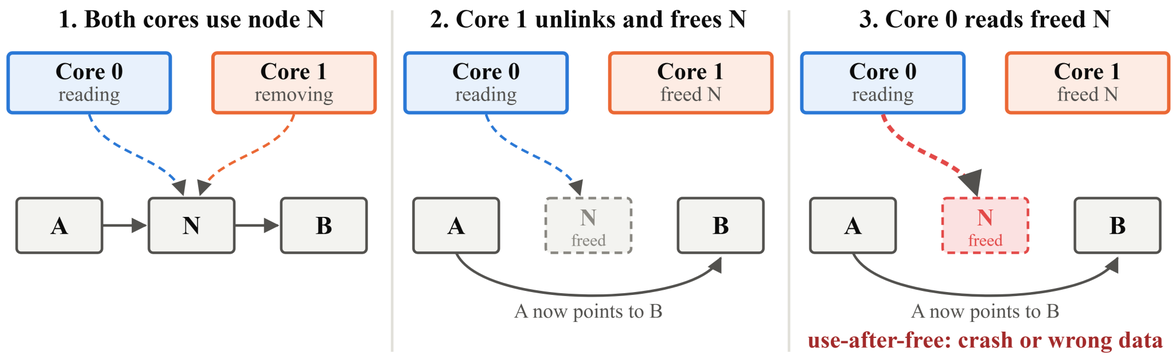


Figure 1. A use-after-free on a shared linked list.

2 Existing software approaches

- **Hazard pointers (HP)** [3]: a reader announces each pointer before using it, and a node is freed only when no announcement names it. *Problem:* every read pays a memory fence, which slows traversals.
- **Epoch-based reclamation (EBR)** [4]: removed nodes are freed in batches once every thread has passed a quiescent point. *Problem:* unfreed memory piles up, without bound if one thread stalls.

The common issue: delayed reclamation. Both schemes free a node only some time after it is removed, so they trade speed against memory.

3 Solution: Conditional Access

Conditional Access (CA) [1] proposes new memory-access instructions for the ISA. It is a hardware–software co-design: a small hardware addition captures cache-coherence events at the cache, and the new instructions expose them to programs, which use them to rule out use-after-free. A thread *watches* the lines it reads; if another core writes one of them, its next CA instruction fails and it retries (Figure 2). Threads can therefore reclaim nodes **immediately**: CA gives the memory footprint of a sequential data structure with the throughput of a concurrent one, and reuses the hardware's own synchronization instead of rebuilding it in software.

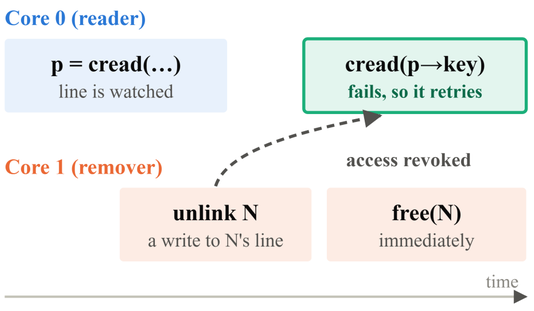


Figure 2. With CA, the remover's write revokes the reader's access, so the reader retries instead of using freed memory.

`cread`	load and watch the line
`cwrite`	store if nothing watched changed
`untagOne`	stop watching one line
`untagAll`	stop watching all lines

```
retry:
  untagAll();
  pred = cread(&head);                  /* read and watch          */
  curr = cread(&pred->next);
  while (cread(&curr->key) < key) {
      if (ca_failed()) goto retry;      /* a watched line changed  */
      pred = curr;  curr = cread(&curr->next);
  }
```

Listing 1. Searching a lazy list with CA, simplified from [1].

4 Why implement it on an FPGA?

CA was evaluated only on a simulator, which cannot give a high-fidelity picture of its real performance impact or of the hardware it needs. Implementing it on an FPGA moves CA from an algorithmic idea to a working hardware mechanism, and tests whether its benefits survive, or become more compelling, in real hardware. It answers questions a simulator cannot:

- **Performance:** the real latency of the CA instructions and the throughput of data structures that use them (Table 2).
- **Coherence overhead:** what the coherence traffic CA depends on actually costs (Table 2).
- **Hardware cost:** the area, logic and memory the mechanism requires (Table 2).

5 System design

We started from CVA6 [2], an open-source single-core RISC-V processor, and added:

(1) a second core with its own private data cache;
(2) MSI cache coherence between the two caches, which CA relies on;
(3) the four CA instructions in the processor pipeline.

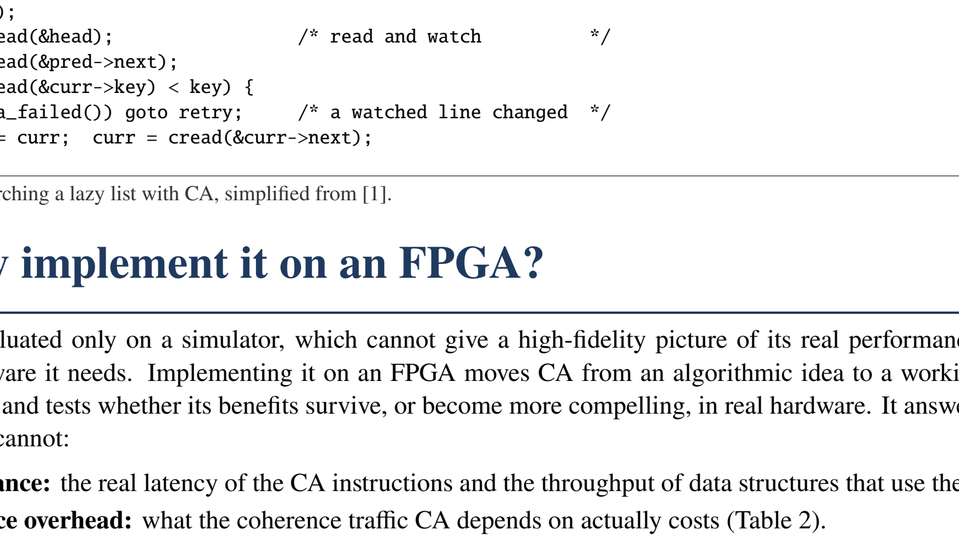


Figure 3. The two-core system on the AMD KR260. Blocks marked *new* were designed in this work.

Each cache line is Modified, Shared or Invalid. A cache takes the shared bus before touching memory and the other cache writes back dirty data first, so memory stays current and no L2 is needed. Programs run bare metal: the host loads them over JTAG and reads results from an uncached mailbox.

6 Challenges

Choosing a base. We first tried OpenPiton, the usual route to a multicore CVA6. It is a heavy framework built around an older CVA6: we would have had to bring it up to date with current CVA6 before adding our changes, including a new L1.5 adapter. We dropped it and built a coherent two-core system directly on current CVA6.

Fitting the FPGA. The KR260's limited LUTs made it hard to fit a two-core 64-bit system with room for our tests, so we moved to the 32-bit (RV32) CVA6 and designed the caches, coherence and tests around it. The final system uses 50% of the LUTs and 51% of the block RAM, and meets timing at 50 MHz; each CA unit tracks 8 lines.

Deadlock freedom. A cache can always answer the other cache's coherence request without needing the shared bus itself, so one of the two always makes progress.

Atomic `cwrite`. The check and the store must be indivisible. `cwrite` waits until it is the oldest instruction and holds the cache line exclusively, so no other core can write between the check and the store.

Speculation. The pipeline issues instructions before it knows they will complete. CA operations are recorded at issue and take effect only at commit, so a cancelled instruction leaves no trace.

Learning curve. We had to learn CVA6's microarchitecture, the KR260 and its Vivado flow, and the underlying concepts: cache coherence, RISC-V atomics and lock-free memory reclamation.

7 Verification

We verified at three levels: Verilator testbenches for one and two caches driven by random traffic, the full processor running standard RISC-V test suites, and two-core programs on the board.

- 1,100 of 1,100 randomized simulations pass against models of MSI and of CA [1].
- 26 of 27 deliberately injected bugs are detected; the remaining mutant cannot change behaviour with two caches.
- 72 of 73 RISC-V ISA tests pass in simulation and on the FPGA, and the two-core coherence and CA programs pass 20 of 20 board runs.
- The testbenches found two real races in our first design (a stale shared copy during an upgrade, and an LR/SC livelock between the cores), both fixed.

Negative control: with ordinary loads the board program reads freed memory; with `cread` it never does.

8 Evaluation

We ran the benchmarks of [1] (lazy list, external BST, Treiber stack, hash table) on both cores at 50 MHz, bare metal, with 3,000 operations per thread, 0, 10 and 100% updates and 3 seeds: 288 runs, each followed by a check of the data structure. Table 1 compares our platform with the paper's.

Table 1. Evaluation platform: the paper and this work.

	Singh et al. [1]	This work
Platform	Graphite simulator	AMD KR260 FPGA, 50 MHz
Cores	1 to 32	2 (CVA6, RV32)
Coherence	directory MSI	snooping MSI on a shared bus
Cache line, L2	64 B, 256 KiB shared L2	16 B, no L2

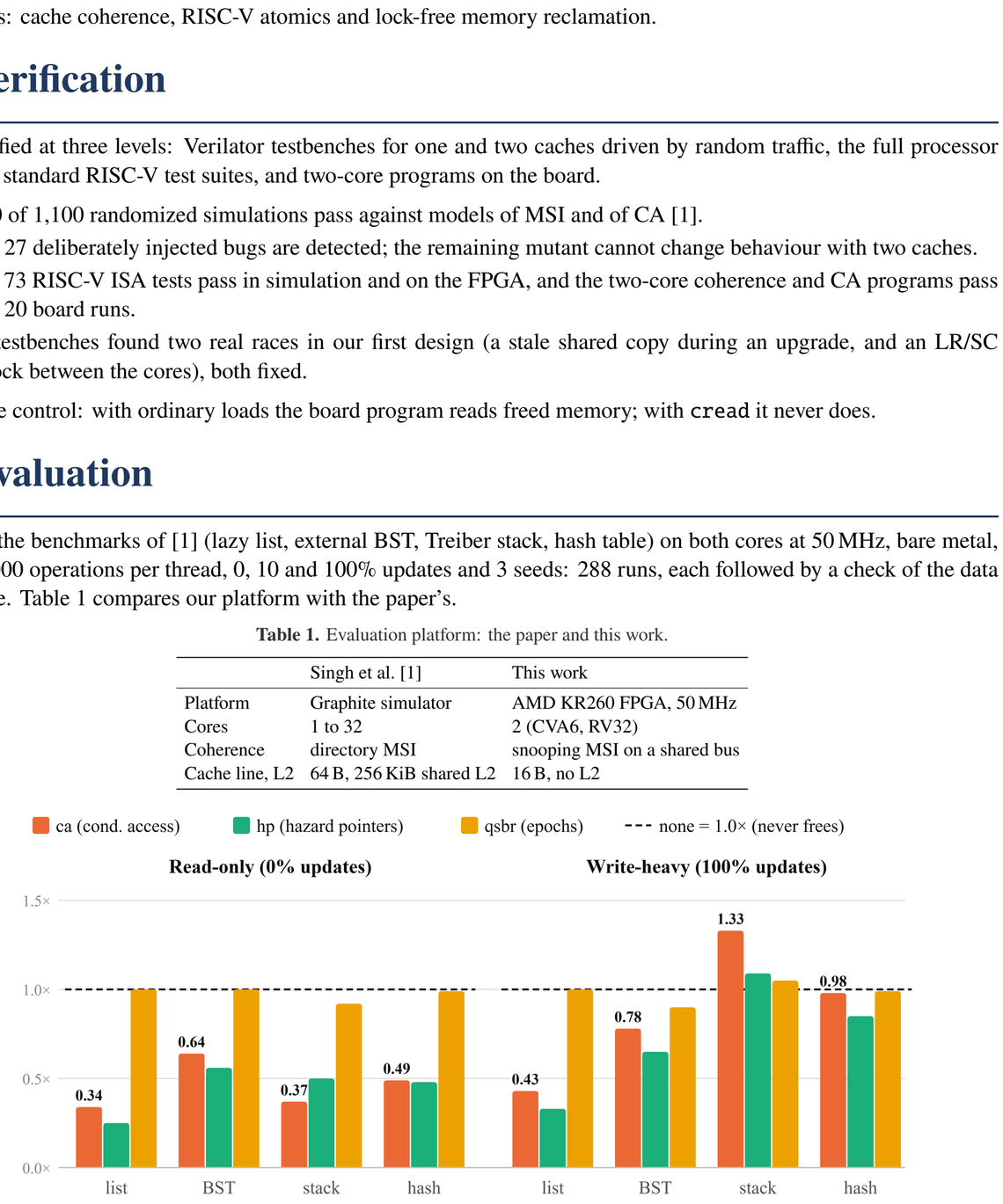


Figure 4. Throughput relative to never freeing memory (higher is better), two cores. Bars above the dashed line outperform the leaking baseline.

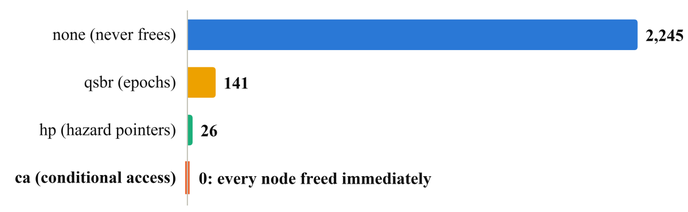


Figure 5. Nodes removed but not yet freed at the end of a run, averaged over the four data structures (two cores, 100% updates).

- Under the highest contention, CA is the fastest scheme: 1.33× the leaking baseline on the stack, against 1.09× for hazard pointers and 1.05× for epochs.
- From 1 to 2 threads on that stack, the leaking baseline falls from 0.36 to 0.19 Mops/s, while CA goes from 0.40 to 0.25 and stays ahead.
- CA frees every node immediately; the other schemes leave tens to thousands of nodes waiting.
- In read-mostly workloads CA is slower, as in [1]: the cost is the failure check (a status-register read at commit), not `cread` itself (Table 2).

Table 2. What the FPGA measured: answers to the questions in Section 4.

Question	Measured on the KR260
Latency	load 1 cycle; `cread` 2; `cread` + failure check 7; `cread` + `cwrite` 17 (`amoswap` 14; hazard-pointer fence 6)
Throughput	up to 1.33× the leaking baseline under contention (Figure 4); 0 nodes left unfreed (Figure 5)
Coherence overhead	read miss 54 cycles, 97 if the other core holds the line dirty; invalidating a clean remote copy adds about 1 cycle; core-to-core handoff 151 cycles
Area, logic, memory	CA unit 1,139 LUTs + 538 flip-flops per core (4.5% of a core, 1% of the FPGA), no block RAM; coherence bus 48 LUTs + 40 flip-flops

9 Conclusions

We built and verified a dual-core RISC-V processor with Conditional Access and the cache coherence it depends on, running on an FPGA. It reclaims memory immediately and is the fastest scheme under contention, confirming the findings of [1] on real hardware. Future work: a branch-on-failure instruction to make the check cheap, and more cores with a scalable interconnect.

References. [1] A. Singh, T. Brown, M. Spear. Efficient hardware primitives for immediate memory reclamation in optimistic data structures. arXiv:2302.12958, 2023. [2] F. Zaruba, L. Benini. The cost of application-class processing. *IEEE TVLSI*, 2019. [3] M. M. Michael. Hazard pointers: safe memory reclamation for lock-free objects. *IEEE TPDS*, 2004. [4] T. E. Hart et al. Performance of memory reclamation for lockless synchronization. *JPDC*, 2007.

Acknowledgements. We thank Prof. Ajay Singh for his guidance throughout this project, and the Department of Computer Science and Engineering, IIT Gandhinagar, for access to the AMD KR260 board and tools.